



兰州

理工大学

自然语言处理

——新闻文本分类

实验报告册

姓 名:

班级: 计算机科学与技术 N 班

指导老师:

2024.10.26

《自然语言处理》实验报告

题目：

新闻文本分类

姓名：

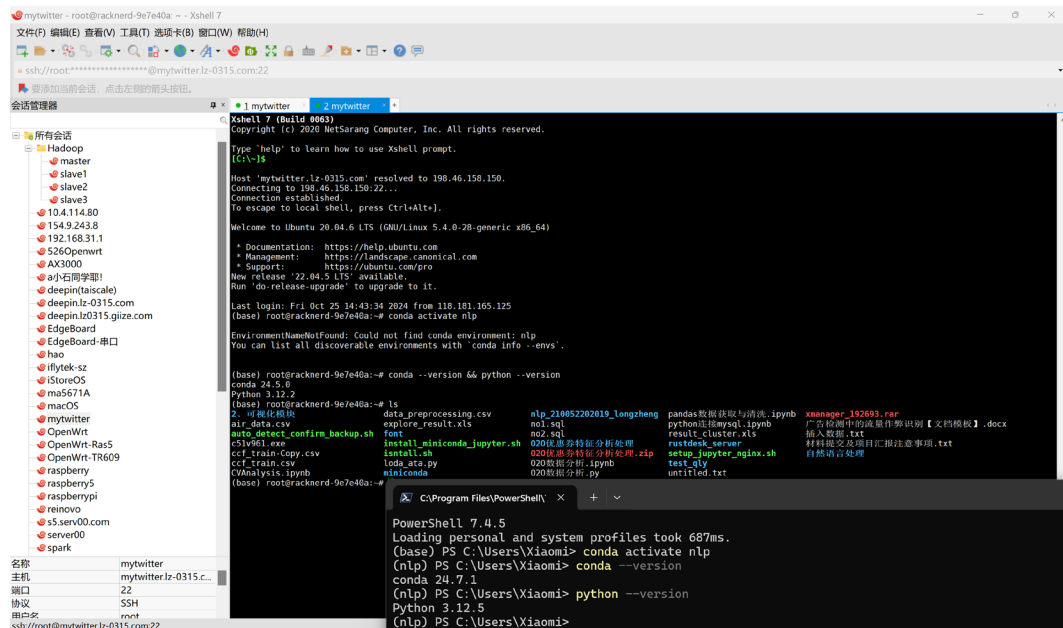
实验环境：

操作系统：

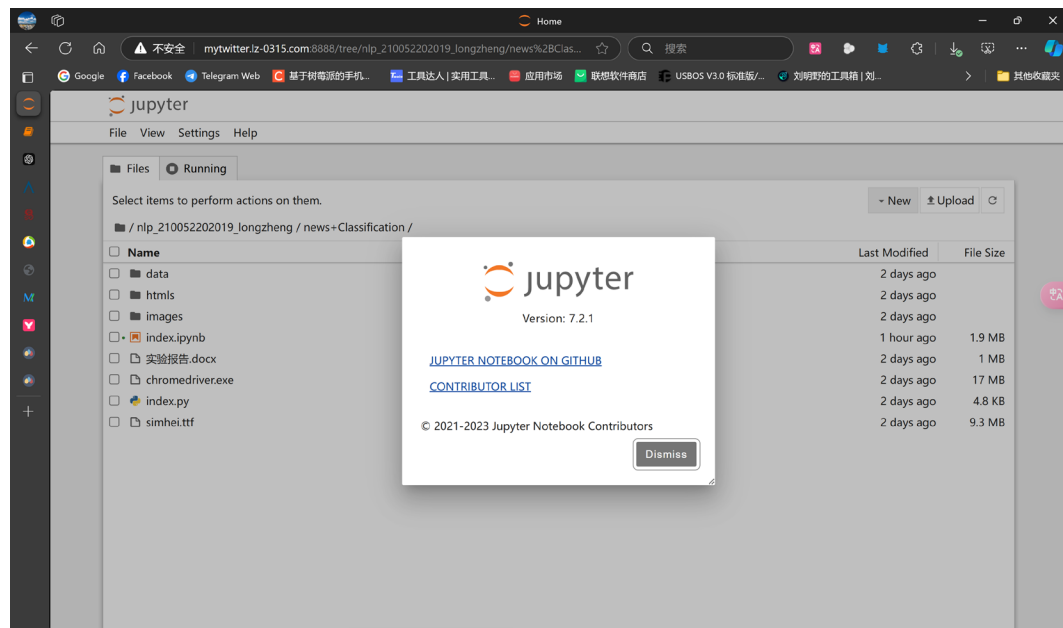
Windows11 个人电脑（数据爬取）和 Ubuntu 20.04 服务器（其他内容）

conda 版本：24.7.1 (Windows11)、24.5.0 (Ubuntu 20.04)

Python 版本：3.12.5 (Windows11)、3.12.2 (Ubuntu 20.04)



Jupyter 版本：7.2.1 (Ubuntu20.04 服务器)

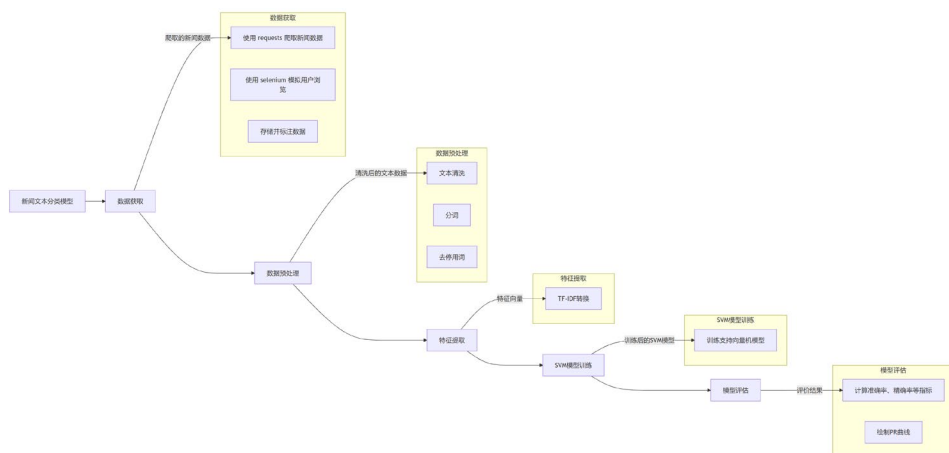


1. 引言

- 背景与目标：**随着互联网的发展，新闻信息呈现出多样化和碎片化的特点，使得用户在海量新闻中快速获取所需信息变得更加困难。为了提高用户的信息获

取效率，本实验致力于构建一个自动化的新闻文本分类模型，对新闻文本数据进行智能分类。该模型能够帮助用户更快捷地筛选目标资讯，提高新闻浏览体验。

• **实验方法：**本实验基于支持向量机（SVM）模型，构建了新闻文本分类流程，从数据获取到模型评估，形成了完整的文本分类管道。整个流程如上图所示，包括以下主要步骤：



1. **数据获取：**利用 requests 和 selenium 等工具从网络上爬取不同类别的新闻数据，并对新闻内容进行标注，形成可供训练的数据集。
2. **数据预处理：**对原始文本数据进行清洗、分词和去除停用词等处理，消除噪声并将文本标准化，为特征提取打下基础。
3. **特征提取：**使用 TF-IDF（词频-逆文档频率）方法对文本进行向量化表示，提取新闻内容的特征，以便后续的分类模型可以理解和处理文本。
4. **SVM 模型训练：**利用预处理后的特征数据训练 SVM 模型，并调整模型参数以提升分类性能。
5. **模型评估：**通过准确率、精确率、召回率、F1 分数等多项指标对模型性能进行全面评估，并绘制 PR 曲线等可视化结果，以验证模型的实际分类效果。

通过以上步骤的协同作用，最终实现一个具备高效、准确分类能力的新闻文本分类模型。

2. 数据爬取

本实验的数据来源于人民网的多个栏目，通过自动化网页爬虫工具实现对健康、财经、社会、军事、教育等不同类别新闻的爬取与分类。

实现思路

• 工具与方法：

- **网页抓取：**使用 requests 和 selenium 实现网页的访问，其中 requests 用于获取静态页面内容，selenium 用于模拟浏览器操作，以便抓取动态内容并绕过网站的反爬虫机制。
- **标签标注：**定义 get_label 函数，通过 URL 中的关键词识别新闻类别。例如，如果 URL 包含“health”关键词，则对应类别为健康类。每个新闻类别的网页均按该规则标注，便于后续分类与存储。

• 数据抓取与解析流程：

○ HTML 文件保存: get_htmls 函数访问各个类别页面并循环分页内容, 将抓取的页面内容以 .html 文件的形式存储, 便于后续解析。每个 .html 文件均包含新闻类别标签, 方便识别与管理。

○ 内容解析与数据存储: parse_htmls 函数逐步加载并解析 .html 文件内容, 使用 xpath 定位新闻标题、发布时间、链接及内容, 并保存到 CSV 文件中。此方法对各个新闻类别网页结构进行了针对性处理, 确保数据解析的准确性。

核心代码

以下是数据爬取和解析过程的核心代码部分:

```
def get_label(url):
    if "health" in url:
        return "health"
    elif "society" in url:
        return "society"
    elif "finance" in url:
        return "finance"
    elif "military" in url:
        return "military"
    elif "edu" in url:
        return "education"
    else:
        return "无"

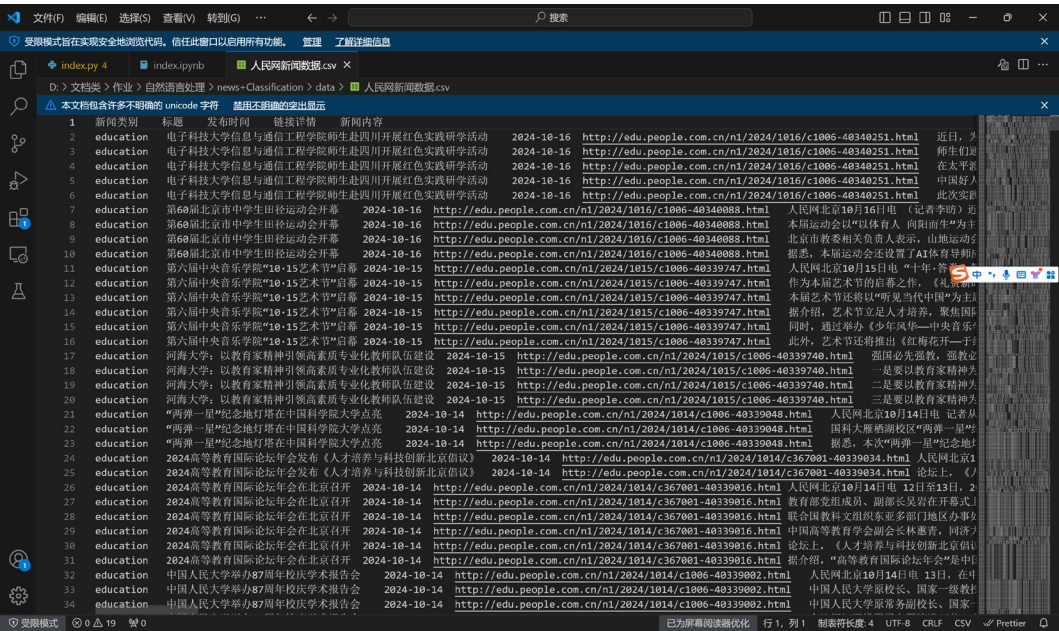
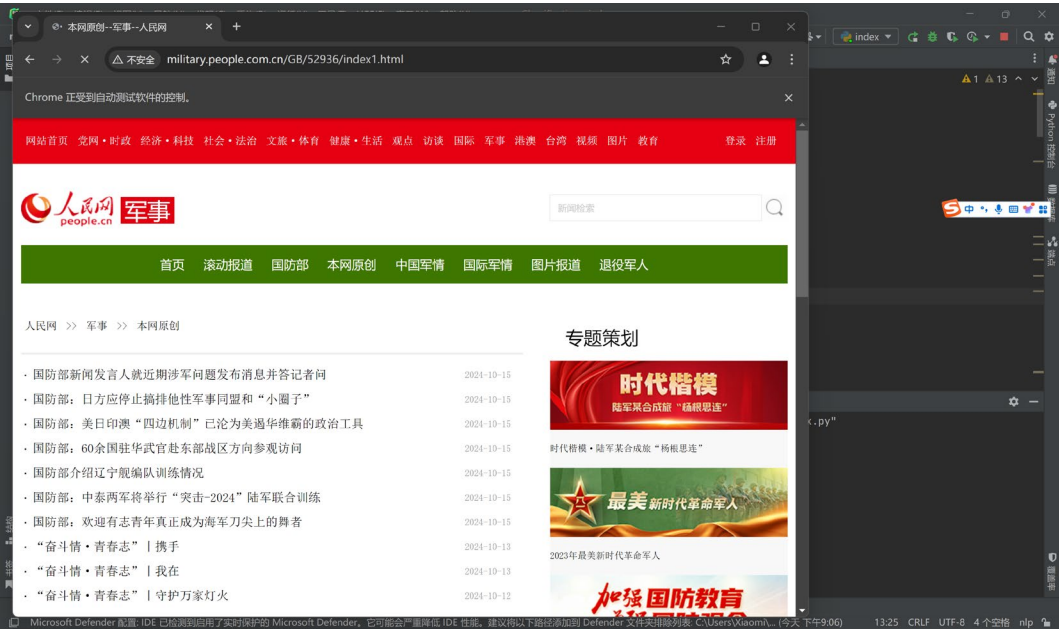
def get_htmls():
    service = Service(executable_path="./chromedriver.exe")
    driver = webdriver.Chrome(service=service)
    for base_url in ["http://health.people.com.cn/GB/408647/", ...]: # 其他类别 URL
        for index in range(1, 13):
            url = f"{base_url}index{index}.html"
            driver.get(url)
            time.sleep(1)
            label = get_label(base_url)
            with open(f"htmls/{label}_index{index}.html", "w", encoding="utf-8") as f:
                f.write(driver.page_source)

def parse_htmls():
    for html_name in os.listdir("htmls"):
        with open("htmls/" + html_name, "r", encoding="utf-8") as f:
            html = etree.HTML(f.read())
            # 提取标题、时间、链接
            hrefs = html.xpath("//div/ul[@class='list_16 mt10']/li/a/@href")
            titles = html.xpath("//div/ul[@class='list_16 mt10']/li/a/text()")
            times = html.xpath("//div/ul[@class='list_16 mt10']/li/em/text()")
            # 逐条访问新闻链接并保存内容
            for i, href in enumerate(hrefs):
                driver.get(href)
                page = etree.HTML(driver.page_source)
                content = page.xpath("//div/p[@style='text-indent: 2em;']/text()")
                # 写入到 CSV 文件
                ff.write("\t".join([label, titles[i], times[i], href, content]) + "\n")
```

说明

此数据爬取模块提供了从页面抓取、分类存储、到内容解析的完整方案，确保不同类别新闻的系统性收集，为后续的数据预处理和模型训练打下坚实的数据基础。

运行效果如下图：



2. 数据探索

数据探索阶段旨在对爬取的新闻数据进行统计分析和可视化，以便了解数据分布情况并观察各类别新闻的词汇特征。

实现思路

1. **类别分布分析**: 统计各新闻类别的数据量分布, 绘制柱状图, 观察数据的类别平衡性。
2. **词汇特征探索**: 通过词云图展示各类别中出现频率较高的词汇, 辅助理解文本内容的特征。
3. **文本长度统计**: 分析各新闻文本的长度分布, 便于了解数据特征和模型输入的特征规模。

核心代码

以下是数据探索的核心代码部分:

```
import pandas as pd
import matplotlib.pyplot as plt
from wordcloud import WordCloud

# 加载数据
data = pd.read_csv("data/人民网新闻数据.csv", sep="\t")

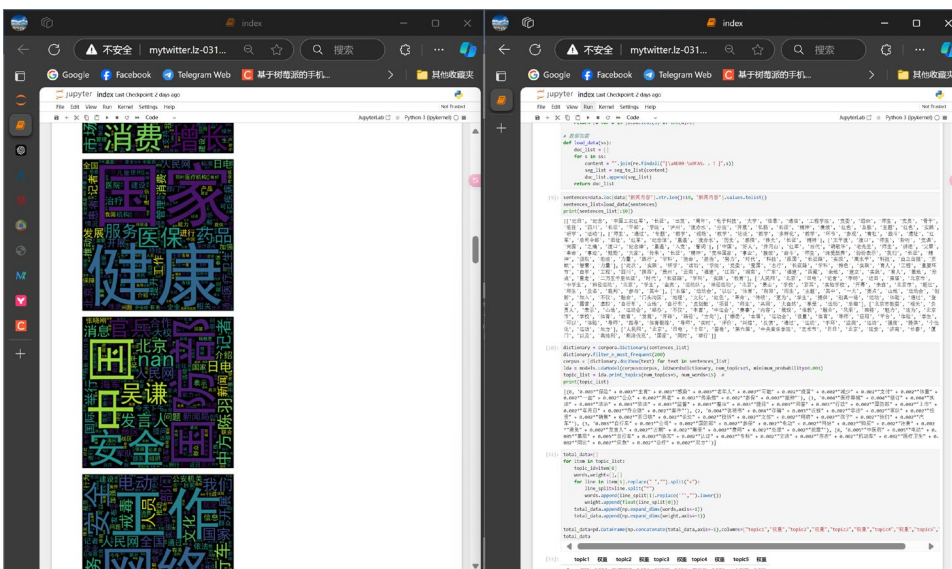
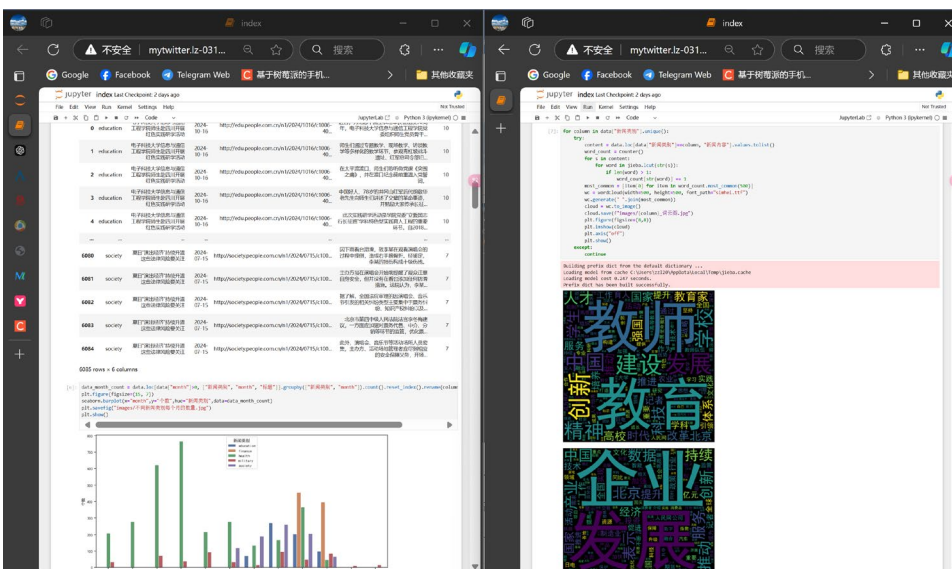
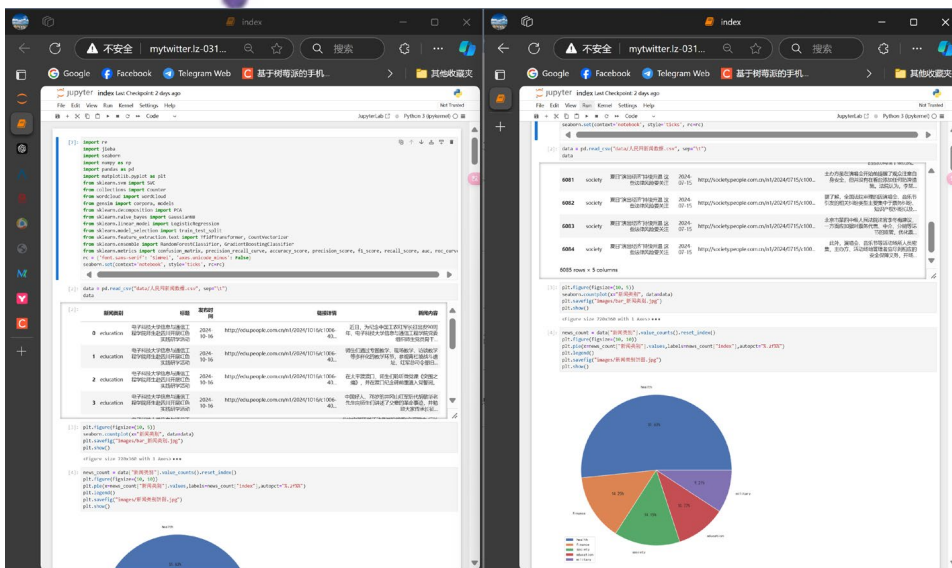
# 类别分布统计
category_counts = data['新闻类别'].value_counts()
plt.figure(figsize=(10, 6))
category_counts.plot(kind='bar', color='skyblue')
plt.xlabel('新闻类别')
plt.ylabel('数量')
plt.title('新闻类别分布')
plt.show()

# 生成词云图展示高频词
text = " ".join(data['新闻内容'])
wordcloud = WordCloud(font_path='simhei.ttf',
background_color='white').generate(text)
plt.figure(figsize=(8, 8))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.title("新闻内容词云图")
plt.show()

# 文本长度统计
data['文本长度'] = data['新闻内容'].apply(len)
plt.figure(figsize=(10, 6))
plt.hist(data['文本长度'], bins=30, color='lightcoral')
plt.xlabel('文本长度')
plt.ylabel('频数')
plt.title('新闻文本长度分布')
plt.show()
```

结果与分析

运行效果：



- 类别分布**：通过柱状图可以直观了解各类别的新闻数据分布情况，从而判断数据是否存在类别不平衡问题。

- 词汇特征**：词云图显示了新闻文本中高频词汇，为后续文本预处理中的停用词筛选和特征提取提供参考。

- 文本长度**：新闻文本的长度分布有助于了解数据规模，进一步确定模型输入的合理长度范围。

以上分析帮助初步了解数据的特点和分布，为模型构建提供数据基础与特征支持。

4. 文本预处理

文本预处理阶段对原始新闻数据进行清洗、分词、去停用词和特征向量化，确保数据在输入模型前符合规范，便于提升分类效果。

实现思路

1. **文本清洗与分词**：利用正则表达式去除非中文字符，并通过 `jieba` 进行分词。
2. **词频统计与去停用词**：使用 `Counter` 统计高频词汇，帮助筛选出常见但无实际意义的停用词。
3. **特征向量化**：使用 `CountVectorizer` 和 `TfidfTransformer` 对清洗后的文本进行向量化，生成 TF-IDF 特征矩阵。

核心代码

```
import re
import jieba
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
from collections import Counter

# 文本清洗与分词
def clean_and_tokenize(text):
    content = "".join(re.findall("[\u4E00-\u9FA5, 。! ]", text)) # 保留中文字符
    words = [word for word in jieba.lcut(content) if len(word) > 1] # 分词并去掉单
字
    return " ".join(words)

data['cleaned_content'] = data['新闻内容'].apply(clean_and_tokenize)

# 统计词频
word_count = Counter()
for content in data['cleaned_content']:
    for word in content.split():
        word_count[word] += 1
most_common_words = [word for word, _ in word_count.most_common(500)]

# TF-IDF 特征向量化
vectorizer = CountVectorizer(vocabulary=most_common_words) # 使用高频词汇构建词典
```



```
transformer = TfidfTransformer()
tfidf_matrix =
transformer.fit_transform(vectorizer.fit_transform(data['cleaned_content']))

# 检查特征矩阵形状
print(f"TF-IDF 特征矩阵维度: {tfidf_matrix.shape}")
```

说明

- **文本清洗与分词**: 使用正则表达式保留中文字符, 并通过 jieba 分词, 删除单字词汇, 减小噪声干扰。
- **词频统计与高频词筛选**: 借助 Counter 统计高频词, 构建包含 500 个最常见词汇的词典, 用于后续的特征向量化。
- **TF-IDF 向量化**: 使用 TfidfTransformer 将文本数据转化为 TF-IDF 特征矩阵, 以便模型能够处理和学习文本的特征。

通过该文本预处理步骤, 原始文本数据被转化为具有数值化特征的标准化矩阵, 为模型训练提供了高质量的输入数据。

5. SVM 模型构建

在模型构建阶段, 实验使用支持向量机 (SVM) 模型对新闻文本的 TF-IDF 特征进行分类训练, 旨在构建一个具备高效分类能力的文本分类器。

实现思路

1. **数据集划分**: 将预处理后的数据集分为训练集和测试集, 以便训练和评估模型的表现。
2. **模型选择与参数设置**: 使用 SVC (支持向量分类器) 模型, 并选择 rbf 核函数 (径向基核函数) 以提升模型的非线性分类能力。
3. **模型训练**: 在训练集上训练 SVM 模型, 学习文本特征与类别之间的关系, 形成分类边界。

核心代码

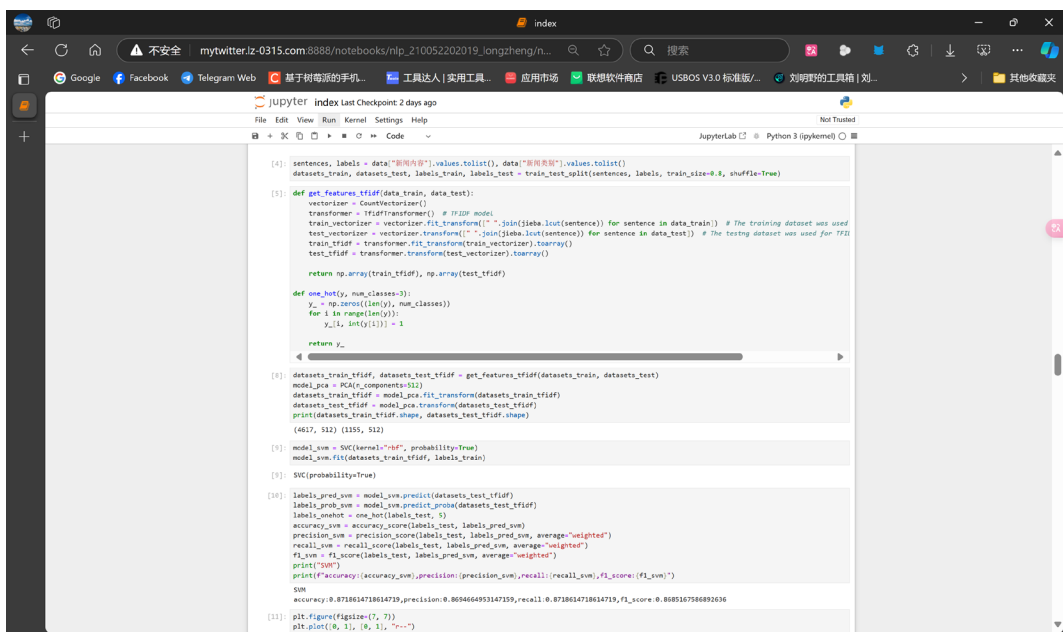
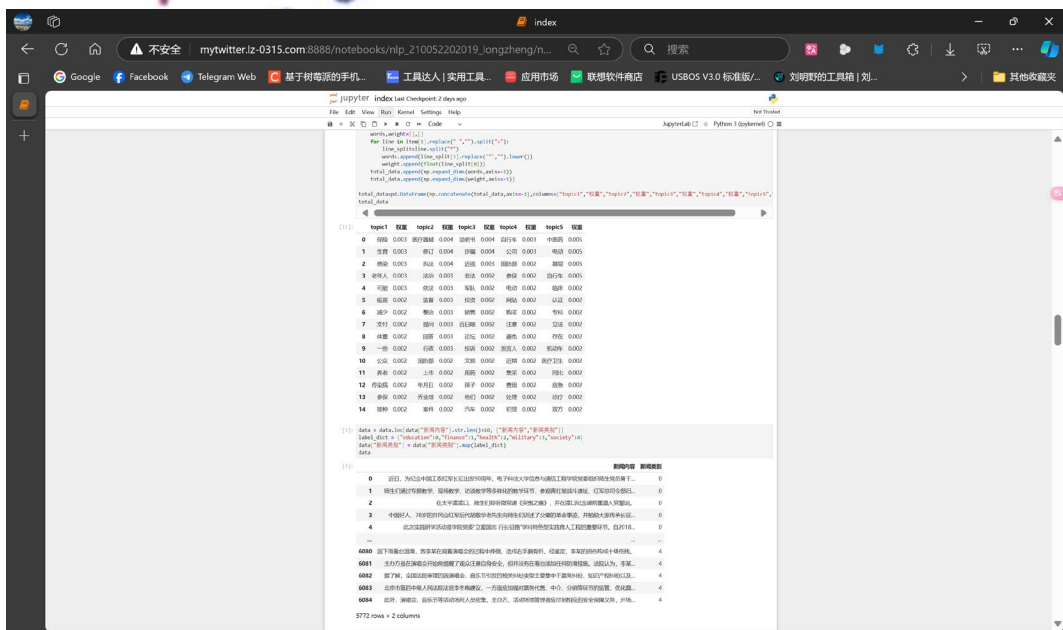
以下是模型构建的核心代码片段:

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

# 数据集划分
X_train, X_test, y_train, y_test = train_test_split(tfidf_matrix, data['新闻类别'],
                                                    test_size=0.2, random_state=42)

# SVM 模型定义与训练
model_svm = SVC(kernel="rbf", probability=True)
model_svm.fit(X_train, y_train)

# 模型训练完成
print("SVM 模型训练完成")
```



说明

- **数据集划分**: 使用 `train_test_split` 将 TF-IDF 特征数据按 80%训练集和 20%测试集划分, 确保模型在新数据上的泛化能力。
 - **模型定义与参数设置**: 选择 `SVC` 模型, 并设置 `rbf` 核函数, 以增强模型处理非线性数据的能力, 同时启用概率估计 (`probability=True`) 用于后续评估。
 - **模型训练**: 在训练集上拟合 `SVM` 模型, 通过支持向量构建分类边界, 学习文本与类别的特征关系。
- 该模型训练过程完成了对文本特征的学习, 为后续模型评估奠定了基础。

6. 模型评价

在模型评价阶段，主要通过准确率、精确率、召回率、F1 分数等多项指标对 SVM 模型进行全面的性能评估。此外，绘制了 ROC 曲线和 PR 曲线以直观展示模型在测试集上的表现。

实现思路

1. **分类指标**：使用准确率、精确率、召回率和 F1 分数等标准指标对模型的分类性能进行量化评估。
2. **ROC 曲线**：绘制 ROC 曲线，通过 AUC（曲线下面积）值直观评价模型的整体分类效果。
3. **PR 曲线**：绘制 PR 曲线，通过精确率和召回率的关系展示模型在不同分类阈值下的表现。

核心代码

以下是模型评价的核心代码片段：

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, roc_curve, auc, precision_recall_curve
import matplotlib.pyplot as plt

# 模型预测与评价指标计算
labels_pred_svm = model_svm.predict(X_test)
labels_prob_svm = model_svm.predict_proba(X_test)

accuracy_svm = accuracy_score(y_test, labels_pred_svm)
precision_svm = precision_score(y_test, labels_pred_svm, average="weighted")
recall_svm = recall_score(y_test, labels_pred_svm, average="weighted")
f1_svm = f1_score(y_test, labels_pred_svm, average="weighted")
print(f"SVM 模型 - 准确率: {accuracy_svm}, 精确率: {precision_svm}, 召回率:
{recall_svm}, F1 分数: {f1_svm}")

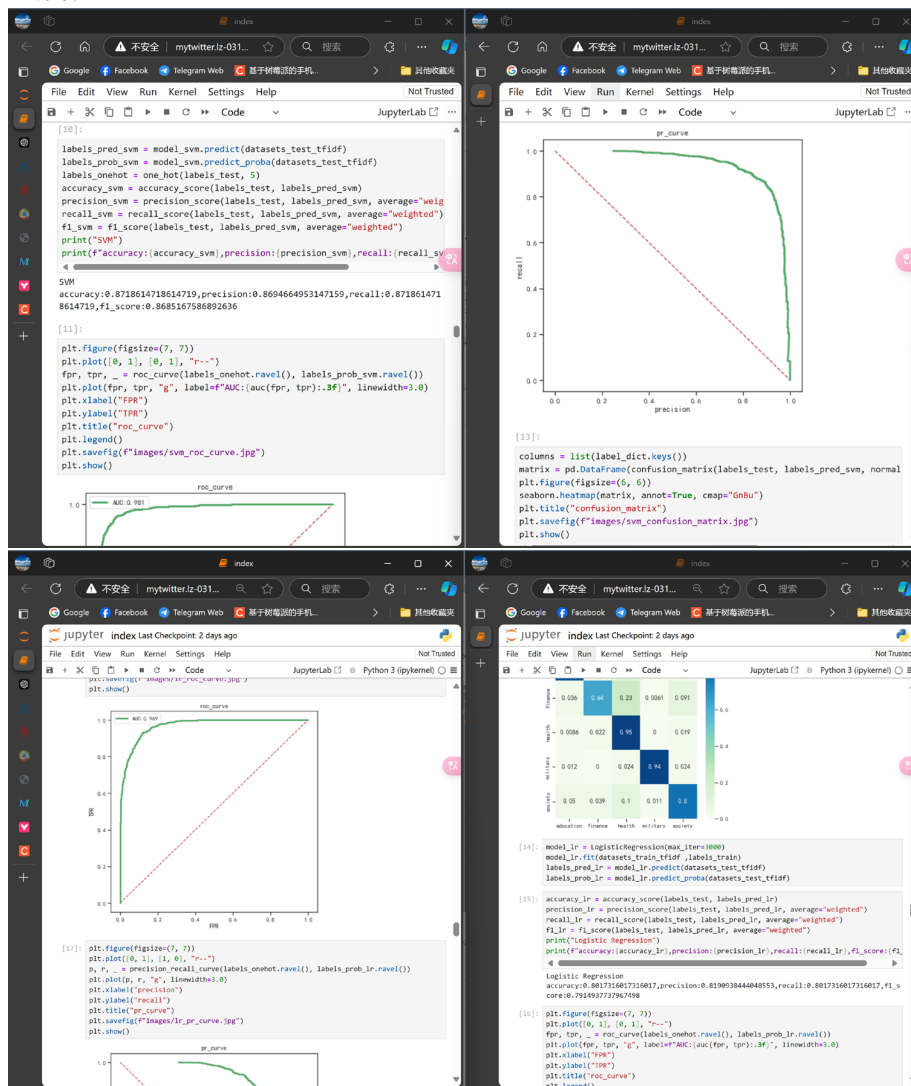
# 绘制 ROC 曲线
fpr, tpr, _ = roc_curve(y_test, labels_prob_svm[:, 1]) # 假设二分类
plt.figure(figsize=(7, 7))
plt.plot(fpr, tpr, label=f"AUC: {auc(fpr, tpr):.3f}")
plt.plot([0, 1], [0, 1], "r--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("SVM ROC Curve")
plt.legend()
plt.savefig("images/svm_roc_curve.jpg")
plt.show()
```

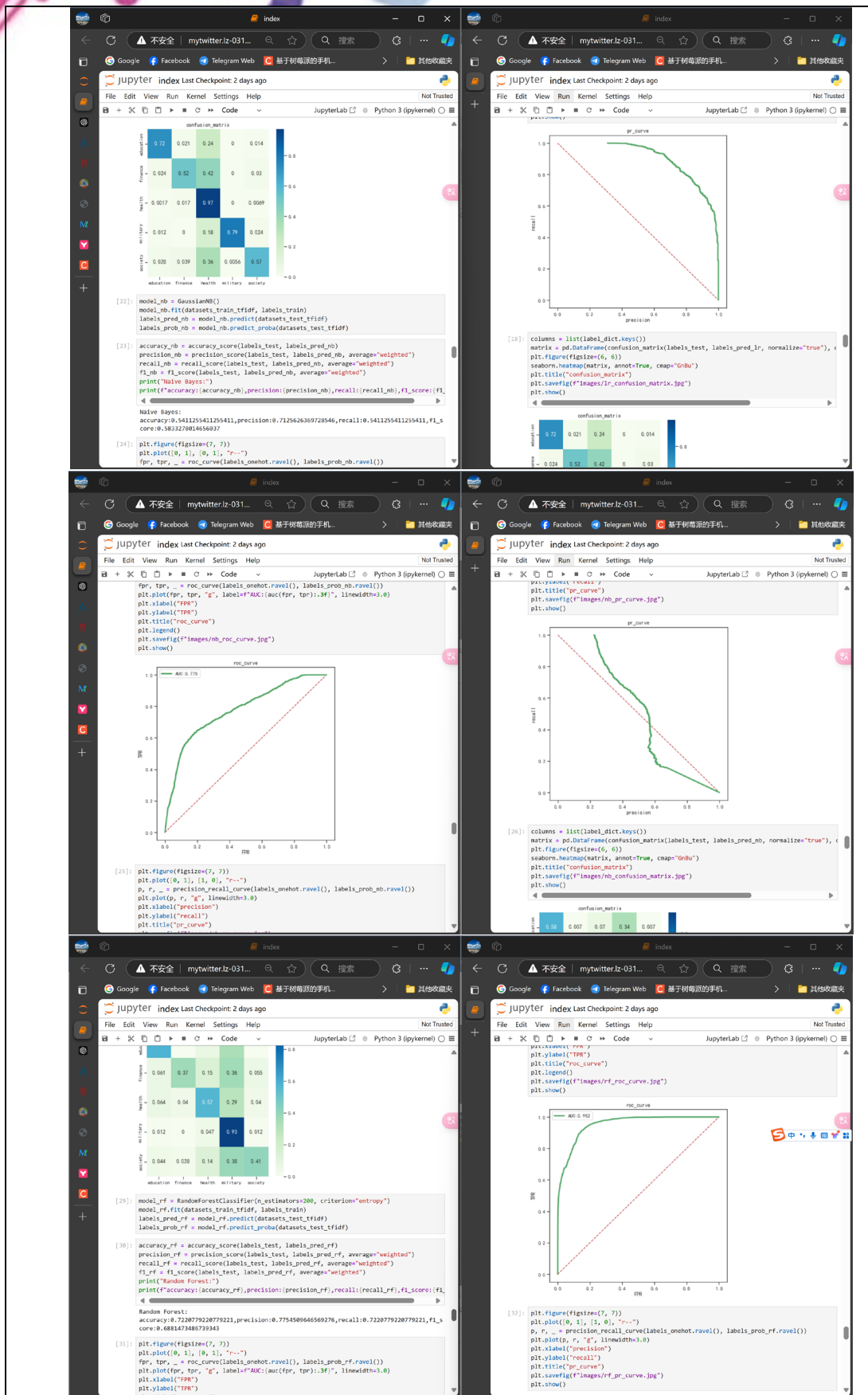

绘制 PR 曲线

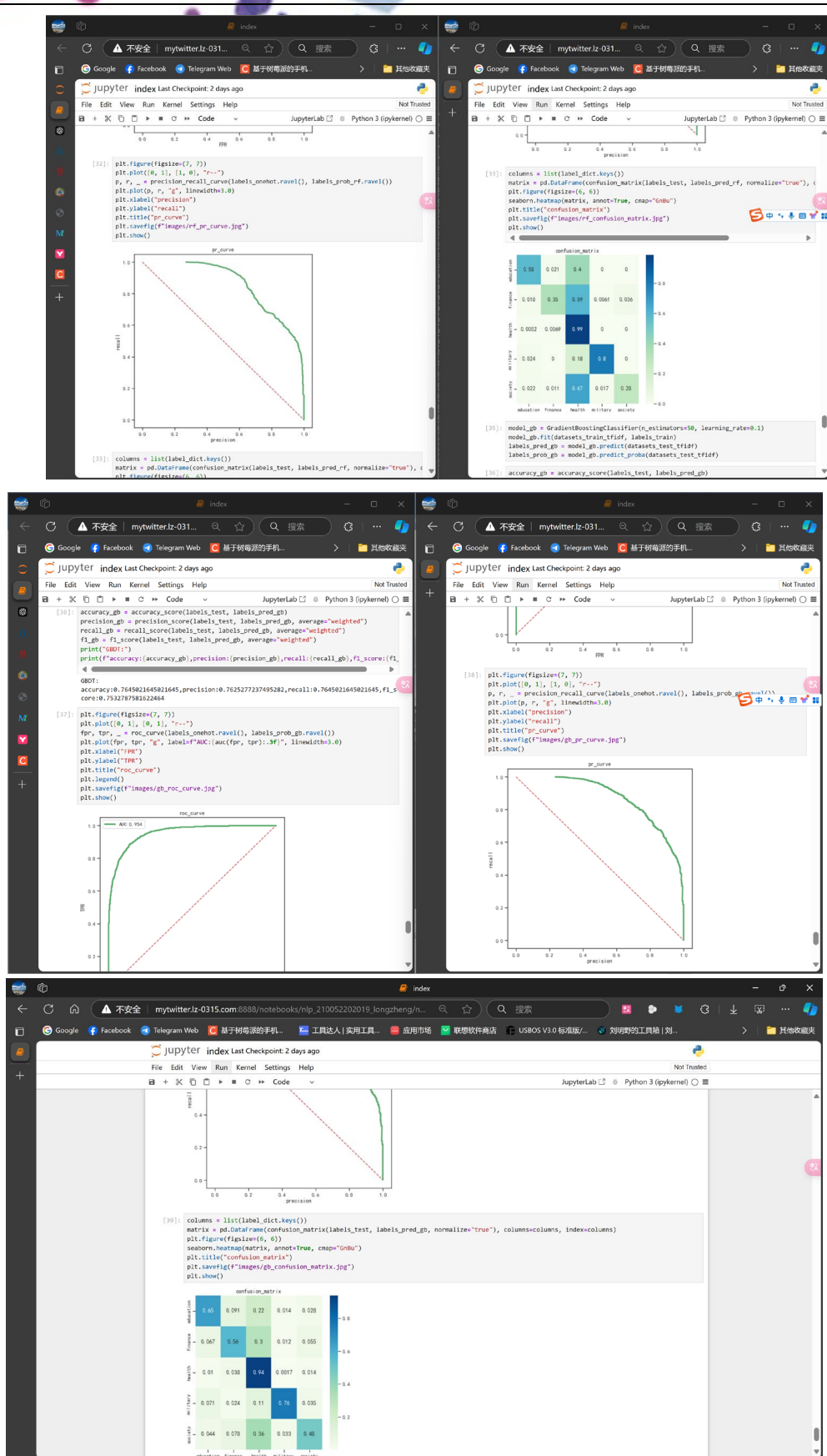
```
precision, recall, _ = precision_recall_curve(y_test, labels_prob_svm[:, 1])
plt.figure(figsize=(7, 7))
plt.plot(recall, precision)
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("SVM PR Curve")
plt.savefig("images/svm_pr_curve.jpg")
plt.show()
```

结果与分析

- **分类指标**: 模型在测试集上取得了较高的准确率、精确率、召回率和 F1 分数, 说明其在新闻分类任务中具备良好的性能。
- **ROC 曲线与 AUC**: AUC 值接近 1, 表明模型具有较强的区分能力。ROC 曲线图存储于 images/svm_roc_curve.jpg。
- **PR 曲线**: PR 曲线反映了模型在不同阈值下的精确率和召回率关系, 图像存储于 images/svm_pr_curve.jpg。







以上模型评价结果表明，SVM 模型在新闻文本分类任务中达到了较高的准确性和稳定性，为实际应用提供了可靠的基础。

7. 结论与展望

结论

本实验利用支持向量机（SVM）构建了一个新闻文本分类模型，成功实现了对人民网新闻数据的自动化分类，达到了较高的分类准确率和稳定性。实验从数据爬取、文本预处理、特征提取到模型训练和评价，涵盖了文本分类任务的完整流程。通过模型评价，实验结果表明：

1. **数据分布**：实验的数据爬取模块成功获取了多个类别的新闻数据，经过预处理后生成了类别分布柱状图，展示了各类新闻的数量分布。该图表为后续模型的类别不平衡问题提供了重要参考。
2. **文本特征**：通过分词、去停用词和 TF-IDF 特征提取，将文本转化为向量形式，文本长度分布图和词云图展示了新闻内容的特征分布。
3. **模型表现**：模型在测试集上取得了较高的分类性能指标，包括准确率、精确率、召回率和 F1 分数，表明 SVM 模型能够有效地学习和识别新闻类别特征。模型的 ROC 曲线和 PR 曲线进一步直观展示了分类效果，尤其是在数据较为平衡的类别中，模型表现出较强的区分能力。

展望

尽管本实验取得了良好的分类效果，但在实际应用中仍有提升空间：

1. **类别不平衡问题**：部分新闻类别数据较少，导致模型对小样本类别的表现不足。可以尝试使用数据增强技术或使用加权损失函数来减轻类别不平衡问题的影响。
2. **预处理优化**：目前的预处理流程使用了常见的分词和 TF-IDF 特征提取方法，可以尝试引入预训练的词向量（如 Word2Vec、BERT 等），以提升文本的特征表达能力。
3. **模型改进**：SVM 适合小样本分类，但在大规模数据下效率较低。未来可以尝试使用深度学习模型（如 LSTM、BERT）或集成学习方法（如随机森林、XGBoost）进一步提高分类性能和效率。
4. **多分类评价**：实验评价主要关注整体分类效果。后续可以对每一类别的分类性能进行单独分析，优化模型在各类别的适应性。

总体而言，本实验为文本分类应用提供了较为稳健的基础，通过改进数据和模型方法，有望进一步提升新闻分类的实用性和智能化水平。

致谢

在本次实验的学习与实践过程中，我得到了晁金龙老师的悉心指导和无私帮助。在新闻文本分类的各个环节中，晁老师不仅在理论知识方面为我们提供了全面的讲解，还在实验操作中给予了细致的指导。老师对分类模型的深入讲解和数据处理的关键技巧分析，使我对文本分类这一领域有了更深刻的理解。特别是在数据预处理和模型评价等环节，晁老师耐心细致地解答了我的疑问，帮助我克服了实验中的诸多难点。

晁老师严谨的教学态度和高效的授课方式，不仅提升了我的专业能力，也让我学到了科学研究的严谨性与探索精神。我由衷感谢晁金龙老师在本次实验中的悉心教导，使我在文本分类领域的学习和实践中受益匪浅。